

ખૂબ સરસ! હવે આપણે **Section 2: INTERMEDIATE LEVEL (The Coder)** પર પહોંચી ગયા છીએ. અહીંથી તમે શીખશો કે JavaScript નો ઉપયોગ કરીને બ્રાઉઝરમાં દેખાતા વેબપેજને વાઈવ કેવી રીતે મેનીપ્યુલેટ કરવું.

ચાલો શરૂ કરીએ **Section 2: Topic 1 — Document Object Model (DOM) & Browser Environment**.

● Section 2: Topic 1 — Document Object Model (DOM) & Browser Environment

1. DOM Tree Architecture (DOM આર્કિટેક્ચર શું છે?)

જ્યારે બ્રાઉઝર કોઈ HTML પેજ લોડ કરે છે, ત્યારે તે આખા પેજને એક ઓબ્જેક્ટના માળખામાં ફેરવી નાખે છે. આ માળખાને **DOM Tree** કહેવામાં આવે છે [MDN].

આ માળખામાં સૌથી ઉપર window ઓબ્જેક્ટ હોય છે (જે આખું બ્રાઉઝર છે), તેની અંદર document હોય છે (જે તમારું વેબપેજ છે), અને તેની અંદર બધા HTML Elements (જેમ કે <body>, <h1>, <div>) એકબીજાના પેરેન્ટ-ચાઇલ્ડ સંબંધથી જોડાયેલા નોડ્સ (Nodes) તરીકે ગોઠવાયેલા હોય છે [MDN].

2. DOM Searching / Querying (એલિમેન્ટ્સને કેવી રીતે શોધવા?)

વેબપેજના કોઈપણ એલિમેન્ટમાં ફેરફાર કરવા માટે, સૌથી પહેલા JavaScript ને તે એલિમેન્ટ બતાવવો પડે. તેને શોધવાની મુખ્ય રીતો આ મુજબ છે [GeeksforGeeks]:

જૂની પદ્ધતિઓ (Legacy Methods)

- document.getElementById('id'): આપેલા ID વાળા સિંગલ એલિમેન્ટને શોધે છે.
- document.getElementsByClassName('class'): તે ક્લાસ વાળા તમામ એલિમેન્ટ્સનું **HTMLCollection** (એક એરે જેવું વિસ્ટ) રિટર્ન કરે છે.
- document.getElementsByTagName('h1'): આપેલા ટેગ વાળા બધા એલિમેન્ટ્સનું વિસ્ટ આપે છે.

આધુનિક અને બેસ્ટ પદ્ધતિઓ (Modern CSS Selector Methods) - [MDN]

- document.querySelector('.my-class'): CSS ની જેમ જ કામ કરે છે. તે મેય થતો પહેલો સિંગલ એલિમેન્ટ જ શોધીને આપશે.
- document.querySelectorAll('div.card'): મેય થતા તમામ એલિમેન્ટ્સ શોધીને એક **NodeList** આપે છે (જેના પર તમે લૂપ ફેરવી શકો છો).

3. DOM Traversal (નોડ્સની આજુબાજુ કેવી રીતે ફરવું?)

એકવાર કોઈ એલિમેન્ટ મળી જાય, પછી તેના આધારે તેના પેરેન્ટ કે ચાઇલ્ડ સુધી પહોંચવાની રીતો [JS.info]:

- element.parentElement: તે એલિમેન્ટના ઉપરના પેરેન્ટ એલિમેન્ટ સુધી પહોંચાડે છે.
- element.children: એલિમેન્ટની અંદર રહેલા તમામ ચાઇલ્ડ એલિમેન્ટ્સનું વિસ્ટ આપે છે.
- element.closest('.container'): આ ખૂબ પાવરફુલ છે! તે ઉપરની તરફ (Upwards) સર્ચ કરીને મેય થતો સૌથી નજીકનો પેરેન્ટ એલિમેન્ટ શોધી આપે છે [MDN].

4. Content & Style Manipulation (ટેક્સ્ટ અને ડિઝાઇન બદલવી)

ટેક્સ્ટ બદલવા માટે: [W3Schools]

- element.innerHTML: આ એલિમેન્ટની અંદરના નવા HTML કોડને પણ સમજીને રેન્ડર કરે છે (સિક્યોરિટી માટે આનો વધુ પડતો ઉપયોગ ટાળવો જોઈએ).
- element.textContent: તે માત્ર પ્લેન ટેક્સ્ટ (Plain Text) બદલવા કે રીડ કરવા માટે બેસ્ટ અને સેફ ઓપ્શન છે [MDN].

સ્ટાઇલ બદલવા માટે: [W3Schools]

javascript

```
const heading = document.querySelector('h1');
```

```
// ૧. ડાયરેક્ટ ઇનલાઇન સ્ટાઇલ આપવી
```

```
heading.style.color = 'blue';
```

```
heading.style.backgroundColor = '#f4f4f4'; // કેમલકેસ (camelCase) વાપરવું પડે
```

```
// ૨. CSS વેરિએબલ (Custom Property) બદલવી
```

```
heading.style.setProperty('--main-theme-color', 'red');
```

Use code with caution.

5. Attributes & Classes Control (ક્લાસ અને એટ્રિબ્યુટ્સ મેનેજ કરવા)

ઇન્ટરેક્ટિવ ડિઝાઇન બનાવવા માટે CSS ક્લાસ એડ કે રીમુવ કરવા ખૂબ જરૂરી છે [MDN]:

javascript

```
const card = document.querySelector('.product-card');
```

```
// Classes મેનેજ કરવા (classList): [MDN]
```

```
card.classList.add('active'); // ક્લાસ ઉમેરશે
```

```
card.classList.remove('hidden'); // ક્લાસ હટાવશે
```

```
card.classList.toggle('dark-mode'); // જો ક્લાસ હશે તો હટાવશે, નહીં હોય તો ઉમેરશે! (ઓટોમેટીક સ્વીચ)
```

```
// Attributes મેનેજ કરવા: [MDN]
```

```
const link = document.querySelector('a');
```

```
link.setAttribute('href', 'https://google.com'); // લિંક બદલવી
```

```
console.log(link.getAttribute('href')); // લિંક ચેક કરવી
```

Use code with caution.

6. Dynamic DOM Creation (નવા એલિમેન્ટ્સ જાતે બનાવવા)

તમારા પેજ પર ડાયનેમિકલી નવી આઇટમ્સ (જેમ કે પ્રોડક્ટ્સ લિસ્ટ) ઉમેરવા માટે નીચેની રીત વપરાય છે [JS.info]:

javascript

```
// ૧. નવો <div> એલિમેન્ટ બનાવો
```

```
const newCard = document.createElement('div');
```

```
// ૨. તેમાં ક્લાસ અને કન્ટેન્ટ ઉમેરો
```

```
newCard.classList.add('mind-item');
```

```
newCard.textContent = "નવી ફૂડ આઇટમ!";
```

```
// ૩. તેને વેબપેજ પર અગાઉથી રહેલા કોઈ પેરેન્ટ એલિમેન્ટમાં ઉમેરો (Append)
```

```
const container = document.querySelector('.mind-flex');
```

```
container.appendChild(newCard); // આ લાઇન રન થતાં જ સ્ક્રીન પર નવો કાર્ડ દેખાશે!
```

Use code with caution.

હવે તમે વેબપેજના એલિમેન્ટ્સ શોધવા, તેની સ્ટાઇલ, ટેક્સ્ટ અને ક્લાસ બદલવા તેમજ નવા એલિમેન્ટ્સ ડાયનેમિકલી બનાવતા શીખી ગયા છો.

હવે આપણે **Section 2: Topic 2 — Advanced Event Architecture (ક્લિક બબ્બલિંગ, Event Bubbling, અને તમારા સ્વાઇડર પ્રોજેક્ટ માટે જરૂરી Event Delegation)** તરફ આગળ વધીએ?

ખૂબ સરસ! ચાલો હવે આગળ વધીએ **Section 2: Topic 2 — Advanced Event Architecture** તરફ.

આ સેક્શનમાં આપણે શીખીશું કે યુઝર જ્યારે પેજ પર કોઈ એક્શન કરે (જેમ કે ક્લિક, ટાઇપિંગ કે સ્ક્રોલ) ત્યારે JavaScript તેની સાથે કેવી રીતે ઇન્ટરેક્ટ કરે છે. આ ટોપિક ઇન્ટરવ્યુ અને તમારા પ્રોડક્ટ સ્વાઇડર બંને માટે મોસ્ટ ઇમ્પોર્ટન્ટ છે [MDN].

● Section 2: Topic 2 — Advanced Event Architecture

1. Event Registration (ઇવેન્ટ જોડવાની રીતો)

વેબપેજ પર કોઈ એલિમેન્ટ સાથે ઇવેન્ટ જોડવાની મુખ્ય ૩ રીતો છે [MDN]:

javascript

```
const btn = document.querySelector('.order-btn');
```

```
// રીત ૧: Inline HTML (આ જૂની રીત છે અને આનો ઉપયોગ ટાળવો જોઈએ)
```

```
// <button onclick="alert('Clicked!')">Order</button>
```

```
// રીત ૨: Element Property (આમાં એક એલિમેન્ટ પર માત્ર એક જ ફંક્શન જોડી શકાય)
```

```
btn.onclick = function() { console.log("પહેલો લોગ"); };
```

```
// રીત ૩: Modern addEventListener() (આ સર્વશ્રેષ્ઠ અને મોડર્ન રીત છે)
```

```
// આનાથી એક જ એલિમેન્ટ પર એકસાથે મલ્ટિપલ ફંક્શન્સ જોડી શકાય છે
```

```
btn.addEventListener('click', () => {
```

```
    console.log("Modern Click 1!");
```

```
});
```

Use code with caution.

2. The Event Object (e)

જ્યારે પણ કોઈ ઇવેન્ટ ટ્રિગર થાય છે (જેમ કે ક્લિક), ત્યારે JavaScript ઓટોમેટિકલી એક ઓબ્જેક્ટ ક્રિએટ કરે છે જેમાં તે ઇવેન્ટની બધી જ સિકેટ માહિતી હોય છે [MDN]. તેને આપણે સામાન્ય રીતે ફંક્શનમાં પેરામીટર તરીકે e અથવા event નામ આપીને રીડ કરીએ છીએ:

javascript

```
btn.addEventListener('click', (e) => {
```

```
    console.log(e.target);    // અસલી એલિમેન્ટ જેના પર ક્લિક થયું છે
```

```
    console.log(e.type);      // ઇવેન્ટનો પ્રકાર (જેમ કે 'click')
```

```
});
```

```
const form = document.querySelector('form');
```

```
form.addEventListener('submit', (e) => {
```

```
    e.preventDefault(); // બ્રાઉઝરના ડિફોલ્ટ બિહેવિયર (પેજ રીફ્રેશ થવું) ને રોકવા માટે!
```

```
});
```

Use code with caution.

3. Event Propagation: Bubbling vs Capturing

જ્યારે તમે કોઈ યાઇલ્ડ એલિમેન્ટ પર ક્લિક કરો છો, ત્યારે તે ક્લિક માત્ર તેના પર જ નહીં, પણ તેના તમામ પેરેન્ટ એલિમેન્ટ્સ (જેમ કે <div>, <body>, window) પર પણ ટ્રિગર થાય છે. આ પ્રોસેસને **Event Propagation** કહેવાય છે [JS.info -> Bubbling and capturing].

આ પ્રોસેસના બે મુખ્ય ફેઝ (Phase) છે:

text

1. Capturing Phase (ઉપરથી નીચે) —> Window થી શરૂ થઈને યાઇલ્ડ એલિમેન્ટ સુધી જાય છે.
2. Bubbling Phase (નીચેથી ઉપર) —> યાઇલ્ડ એલિમેન્ટથી શરૂ થઈને પાછું છેક Window સુધી જાય છે.

Use code with caution.

Event Bubbling (બબલિંગ - ડિફોલ્ટ બિહેવિયર)

પાણીના પરપોટા (Bubbles) ની જેમ ઇવેન્ટ નીચેથી છેક ઉપર સુધી જાય છે.

javascript

// જો તમે અંદરના બટન પર ક્લિક કરશો તો ત્રણેય લોગ વારાફરતી પ્રિન્ટ થશે!

```
document.querySelector('.child-btn').addEventListener('click', () => console.log('Button Clicked'));
```

```
document.querySelector('.parent-div').addEventListener('click', () => console.log('Div Clicked'));
```

Use code with caution.

Propagation રોકવા માટે:

જો તમે ઇચ્છો કે યાઇલ્ડ પર ક્લિક કર્યા પછી ઇવેન્ટ ઉપરના પેરેન્ટ સુધી ન પહોંચે, તો તમે `e.stopPropagation()` નો ઉપયોગ કરી શકો છો [JS.info -> Bubbling and capturing].

4. Event Delegation Pattern (મોસ્ટ આસ્કડ ઇન્ટરવ્યુ કોન્સેપ્ટ)

ધારો કે તમારી પાસે તમારા સ્વાઇડર (જેમ કે સ્વિગી પ્રોજેક્ટ) માં ૧૦૦ ફૂડ આઇટમ્સ ડાયનેમિકલી એડ થાય છે. જો તમે બધા પર અલગ-અલગ `addEventListener` લગાવશો, તો બ્રાઉઝરની બધી મેમરી વપરાઈ જશે અને પેજ લેગ (Lag) થશે [GeeksforGeeks].

Event Delegation નો નિયમ કહે છે: "બધા યાઇલ્ડ એલિમેન્ટ્સ પર અલગ-અલગ લિસનર મૂકવાના બદલે, તેના **મુખ્ય પેરેન્ટ કન્ટેનર** પર માત્ર ૧ જ લિસનર મૂકો, અને Event Bubbling નો ફાયદો ઉઠાવીને ક્લિક હેન્ડલ કરો." [GeeksforGeeks]

રિયલ પ્રોફેશનલ કોડ એક્ઝામ્પલ:

javascript

// આપણે માત્ર સિંગલ પેરેન્ટ (mind-flex) પર લિસનર મૂકીશું

```
const mindFlex = document.getElementById('mind-flex');

mindFlex.addEventListener('click', (e) => {
  //closest() નો ઉપયોગ કરીને ચેક કરો કે ક્લિક થયેલો એલિમેન્ટ કે તેનો પેરેન્ટ '.mind-item' છે કે નહીં
  const clickedItem = e.target.closest('.mind-item');

  if (clickedItem) {
    console.log("ફૂડ આઇટમ પર ક્લિક થયું!", clickedItem);
    // અહીં તમે લાઇવ કન્ટેન્ટ બદલી શકો છો કે યુઝરને બીજા પેજ પર મોકલી શકો છો
  }
});
```

Use code with caution.

તમે હવે એડવાન્સ ઇવેન્ટ હેન્ડલિંગ, બબલિંગ અને તમારા ડેવલપમેન્ટ પ્રોજેક્ટ્સ માટે સુપર-ઇમ્પોર્ટન્ટ એવા **Event Delegation** પેટર્નને કમ્પીટ સમજી ગયા છો.

હવે આપણે આગળના મહત્વના ટોપિક **Section 2: Topic 3 — ES6+ Advanced Data Methods & Operators (map(), filter(), reduce(), Destructuring અને Spread/Rest Operator)** તરફ કદમ વધારીએ?

ખૂબ સરસ! ચાલો હવે આપણે **Section 2: Topic 3 — ES6+ Advanced Data Methods & Operators**

તરફ આગળ વધીએ.

આ આખો ટોપિક ડેટા હેન્ડલિંગ, એરે મેનીપ્યુલેશન અને મોડર્ન ડેવલપમેન્ટ (જેમ કે React, Vue વગેરે) નો પાયો છે. ઇન્ટરવ્યુમાં આ સેક્શનમાંથી સૌથી વધુ પ્રેક્ટિકલ પ્રશ્નો પૂછાય છે [MDN].

● Section 2: Topic 3 — ES6+ Advanced Data Methods & Operators

1. Destructuring Assignment (ડેટા છૂટો પાડવો)

Destructuring એ Array અથવા Object માંથી ડેટાને ખૂબ જ સરળ સિન્ટેક્સ સાથે બહાર કાઢીને વેરિએબલમાં સ્ટોર કરવાની આધુનિક રીત છે [MDN].

Object Destructuring:

javascript

```
const user = { name: "Raj", age: 22, role: "Ninja" };
```

// જૂની રીત:

```
// const name = user.name; const role = user.role;
```

// આધુનિક રીત:

```
const { name, role, city = "Ahmedabad" } = user; // city અહિયાં Default Parameter છે  
console.log(name, role, city); // અઉટપુટ: Raj Ninja Ahmedabad
```

// નામ બદલવું હોય (Renaming):

```
const { name: fullName } = user;  
console.log(fullName); // અઉટપુટ: Raj
```

Use code with caution.

Array Destructuring:

javascript

```
const fruits = ["Apple", "Banana", "Mango"];
```

```
const [first, , third] = fruits; // વચ્ચે ખાલી જગ્યા છોડીને Banana ને સ્કીપ કર્યો
```

```
console.log(first, third); // અઉટપુટ: Apple Mango
```

Use code with caution.

2. Rest Parameter vs Spread Operator (...)

આ ત્રણ ટપકાં (...) JavaScript માં ક્યાં વપરાય છે તેના આધારે બે અલગ ભૂમિકા ભજવે છે [JS.info]:

A. Spread Operator (ડેટા ફેલાવવો - Copy/Merge માટે)

જ્યારે તમારે કોઈ એરે કે ઓબ્જેક્ટની કોપી કરવી હોય કે મર્જ કરવા હોય ત્યારે તે ડેટાને ખોલી નાખે છે (Spread કરે છે):

javascript

```
const arr1 =;
```

```
const arr2 = [...arr1, 4, 5]; // એરે મર્જ કર્યો -> [1, 2, 3, 4, 5]
```

// Shallow Copy (મેમરી રેફરન્સ તોડવા માટે)

```
const originalObj = { a: 1, b: 2 };
```

```
const clonedObj = { ...originalObj };
```

Use code with caution.

B. Rest Parameter (બાકીનો પ્રોગ્રામ ભેગો કરવા માટે)

જ્યારે ફંક્શનના પેરામીટરમાં છેલ્લે વપરાય, ત્યારે તે આવનારી તમામ આર્ગ્યુમેન્ટ્સને ભેગી કરીને એક અસલી એરે (Array) બનાવી દે છે:

javascript

```
function sumAll(...numbers) { // બધા નંબર્સ એક એરેમાં આવી જશે  
  return numbers.reduce((acc, num) => acc + num, 0);  
}
```

```
console.log(sumAll(10, 20, 30)); // આઉટપુટ: 60
```

Use code with caution.

3. Advanced Array Iterators (એરેની આધુનિક મેથડ્સ)

જૂની for લૂપ વાપરવાના બદલે મોડર્ન JS માં આ ઇટરેટર્સનો ઉપયોગ થાય છે. આ બધી મેથડ્સ પોતાનામાં એક **Callback Function** લે છે [W3Schools / MDN].

javascript

```
const products = [  
  { id: 1, name: "Laptop", price: 50000 },  
  { id: 2, name: "Mobile", price: 15000 },  
  { id: 3, name: "Watch", price: 5000 }  
];
```

// 1. map(): દરેક આઇટમ પર પ્રોસેસ કરીને એક નવો જ એરે રિટર્ન કરે છે

```
const productNames = products.map(item => item.name);  
console.log(productNames); // ["Laptop", "Mobile", "Watch"]
```

// 2. filter(): કન્ડિશન સાચી પડે તેટલી જ આઇટમ્સ વાળો નવો એરે આપે છે

```
const cheapProducts = products.filter(item => item.price < 20000);  
console.log(cheapProducts); // Mobile અને Watch વાળો એરે
```

// 3. find(): કન્ડિશન મેચ થાય તેવી સૌથી પહેલી સિંગલ આઇટમ જ રિટર્ન કરશે

```
const mobile = products.find(item => item.name === "Mobile");
```

```
// 4. reduce(): આખા એરેના ડેટા પર ગણતરી કરીને સિંગલ વેલ્યુ (જેમ કે ટોટલ સમ) આપે છે
// acc = Accumulator (ભેગું કરનાર), curr = Current Item
const totalBill = products.reduce((acc, curr) => acc + curr.price, 0); // 0 એ શરૂઆતની કિંમત છે
console.log(totalBill); // આઉટપુટ: 70000
```

```
// 5. some() અને every():
```

```
const hasExpensive = products.some(item => item.price > 40000); // કોઈ એક પણ છે? -> true
```

```
const allExpensive = products.every(item => item.price > 40000); // બધા જ છે? -> false
```

Use code with caution.

4. Map and Set Structures (નવા મોડર્ન ડેટા સ્ટ્રક્ચર્સ)

ES6 માં ડેટા સ્ટોર કરવા માટે બે શક્તિશાળી ઓબ્જેક્ટ્સ આવ્યા [JS.info -> Map and Set]:

- **Set:** આ એવો એરે જેવો સ્ટ્રક્ચર છે જે પોતાનામાં ક્યારેય **ડુપ્લિકેટ (Duplicate)** કિંમતો રાખતો નથી. બધી વેલ્યુ હંમેશા યુનિક જ હોય છે.

javascript

```
const uniqueNumbers = new Set([1, 2, 2, 3, 4, 4]);
```

```
console.log(uniqueNumbers); // આઉટપુટ: Set(4) {1, 2, 3, 4}
```

Use code with caution.

- **Map:** નોર્મલ ઓબ્જેક્ટમાં કી (Key) હંમેશા સ્ટ્રિંગ જ હોઈ શકે, પણ Map માં કી તરીકે તમે નંબર, ફંક્શન કે આખો ઓબ્જેક્ટ પણ મૂકી શકો છો!

હવે તમે ડેટા મેનીપ્યુલેશનની બધી જ ક્રિયા મેથડ્સ (map, filter, reduce) અને મોડર્ન ઓપરેટર્સના માસ્ટર બની ગયા છો!

આગળ વધવા માટે હવે આપણી પાસે છે **Section 2: Topic 4 — Asynchronous Basics (Callbacks, Timers અને Callback Hell નો પ્રોબ્લેમ)**.

શું આપણે આ અસિંક્રોનસ લર્નિંગ શરૂ કરીએ? જ્યારે પણ રેડી હોવ, "Next" અથવા "હા" લખો!

ખૂબ સરસ! ચાલો હવે આપણે **Section 2: Topic 4 — Asynchronous Basics & Web APIs** તરફ આગળ વધીએ.

JavaScript મૂળભૂત રીતે **Single-threaded** અને **Synchronous** ભાષા છે, જેનો મતલબ એ છે કે તે એક સમયે માત્ર એક જ લાઈન રન કરી શકે છે. પરંતુ રિયલ વર્લ્ડમાં જ્યારે આપણે કોઈ ડેટા સર્વર પરથી લાવવાનો હોય,

ત્યારે જો JS લાઇન રોકીને બેસી રહે તો આખી વેબસાઇટ હેંગ થઈ જાય. આ પ્રોબ્લેમ સોલ્વ કરવા માટે બ્રાઉઝર તેના **Web APIs** નો ઉપયોગ કરીને અસિંક્રોનસ (Asynchronous) બિહેવિયર પૂરું પાડે છે [MDN].

● Section 2: Topic 4 — Asynchronous Basics & Web APIs

1. Browser Timers (setTimeout & setInterval)

બ્રાઉઝર આપણને સમય આધારિત કામ કરવા માટે બે મહત્વના ટાઇમર્સ આપે છે, જે બેકગ્રાઉન્ડ થ્રેડ્સમાં ચાલે છે [MDN]:

A. setTimeout()

કોઈ ચોક્કસ સમય (Milliseconds) પછી કોડને **માત્ર એક જ વાર** રન કરવા માટે.

javascript

```
console.log("શરૂઆત");
```

```
const timerId = setTimeout(() => {  
  console.log("આ લોગ ૨ સેકન્ડ પછી દેખાશે!");  
}, 2000); // 2000 ms = 2 સેકન્ડ
```

```
console.log("અંત");
```

```
// આઉટપુટનો ક્રમ: શરૂઆત -> અંત -> (૨ સેકન્ડ પછી) આ લોગ ૨ સેકન્ડ પછી દેખાશે!
```

```
// જો તમારે ટાઇમર પૂરો થાય એ પહેલાં તેને કેન્સલ કરવો હોય:
```

```
clearTimeout(timerId);
```

Use code with caution.

B. setInterval()

કોઈ ચોક્કસ સમયના અંતરે કોડને **વારંવાર (Continuously)** રન કરવા માટે.

javascript

```
let count = 0;
```

```
const intervalId = setInterval(() => {  
  count++;  
  console.log(`આ લોગ દર સેકન્ડે આવશે. કાઉન્ટ: ${count}`);
```

```
  if (count === 5) {
```

```
clearInterval(intervalId); // ૫ વાર રન થયા પછી લૂપ બંધ થઈ જશે  
}  
, 1000); // દર ૧ સેકન્ડે  
Use code with caution.
```

2. Callback Architecture (કોલબેક આર્કિટેક્ચર)

જ્યારે આપણે કોઈ ફંક્શનની અંદર બીજા ફંક્શનને આર્ગ્યુમેન્ટ તરીકે પાસ કરીએ, અને અંદરનું ફંક્શન પોતાનું કામ પૂરું કરીને પેલા પાસ કરેલા ફંક્શનને પાછું કોલ (Call) કરે, તેને **Callback Function** કહેવાય છે [GeeksforGeeks].

javascript

```
function fetchUserData(userId, callback) {  
    console.log("સર્વર પરથી ડેટા આવી રહ્યો છે...");  
  
    setTimeout(() => {  
        const user = { id: userId, name: "Raj", type: "Ninja" };  
        callback(user); // ડેટા મળ્યા પછી કોલબેક રન થશે  
    }, 1500);  
}
```

// ફંક્શન ચલાવતી વખતે આપણે કોલબેક પાસ કરીએ છીએ

```
fetchUserData(101, (data) => {  
    console.log("ડેટા મળી ગયો:", data); // આઉટપુટ: { id: 101, name: "Raj", type: "Ninja" }  
});
```

Use code with caution.

3. Callback Hell / Pyramid of Doom (મુખ્ય પ્રોબ્લેમ)

જ્યારે આપણે એક પછી એક સિક્વન્સમાં ઘણા બધા અસિંક્રોનસ કામો કરવાના હોય (જેમ કે: પેલા લોગીન કરો -> પછી પ્રોફાઈલ લાવો -> પછી ઓર્ડર લાવો -> પછી પેમેન્ટ કરો), ત્યારે એક કોલબેકની અંદર બીજો, તેની અંદર ત્રીજો એમ કોડ નેસ્ટેડ (Nested) થતો જાય છે [GeeksforGeeks].

આનાથી કોડનો શેપ ત્રિકોણ કે પિરામિડ જેવો થઈ જાય છે, જેને **Callback Hell** કહેવાય છે. આ કોડ વાંચવો અને ડીબગ કરવો અત્યંત મુશ્કેલ છે:

javascript

```
// Callback Hell નું એક ઉદાહરણ:  
loginUser("raj@mail.com", "123", (user) => {  
  getUserProfile(user.id, (profile) => {  
    getUserOrders(profile.id, (orders) => {  
      getOrderDetail(orders[0], (detail) => {  
        console.log("ઊલ્લો ડેટા:", detail);  
        // આ કોડ જમણી બાજુ વધતો જાય છે અને મેનેજ થતો નથી!  
      });  
    });  
  });  
});  
});
```

Use code with caution.

અભિનંદન! તમે **Section 2: INTERMEDIATE LEVEL (The Coder)** ના ચારેય મહત્વના સેક્શન્સ કમ્પ્લીટ કરી લીધા છે. હવે તમને એ પણ ખબર પડી ગઈ છે કે જૂની રીતમાં કયા પ્રોબ્લેમ્સ (Callback Hell) હતા.

હવે આપણે સૌથી રોમાંચક અને ઇન્ટરવ્યુના પાકા ઘર સમાન **Section 3: ADVANCED LEVEL (The Ninja)** તરફ પ્રયાણ કરીએ?

તેનો પહેલો અતિ મહત્વનો ટોપિક છે: **Engine Core & Under the Hood (Execution Context, Stack vs Heap, Hoisting, Scope Chain, અને મહાન Event Loop નો આર્કિટેક્ચર).**